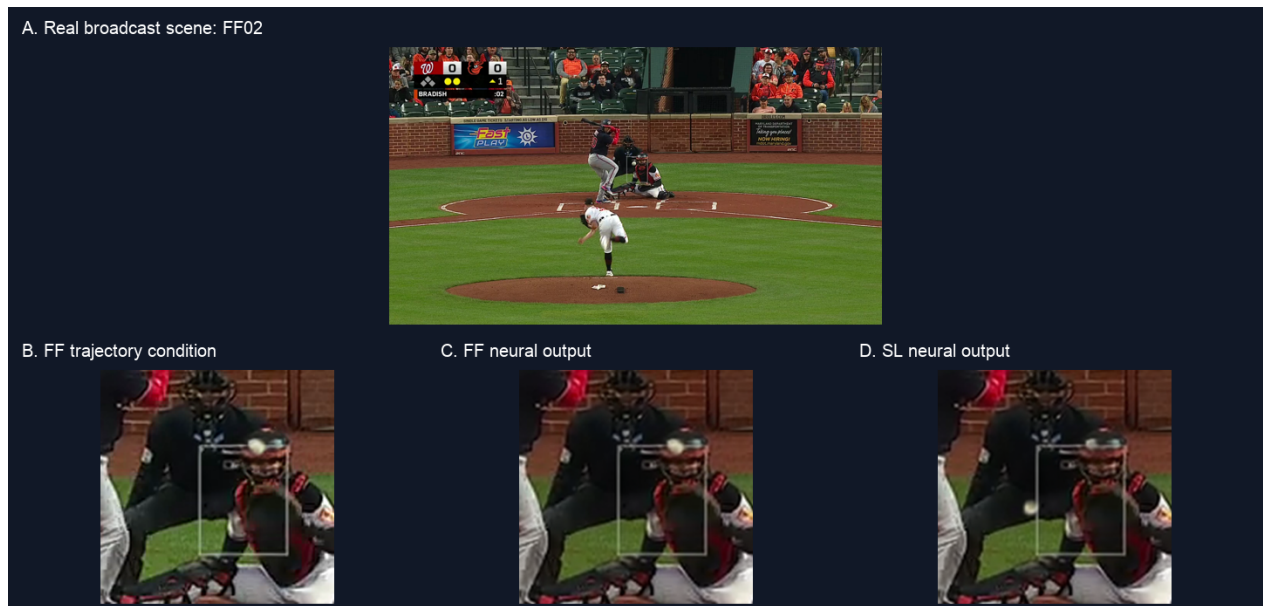


Baseball Trajectory Learning and Controlled Video



Purpose of this document. This is a working project report, not a research-paper claim of validated baseball physics. I record what was built, how it was evaluated, how its code and artifacts are organized, what the current evidence supports, and what the next project phase should investigate.

Contents

1	Introduction and Literature Review	2
2	Scope, assets, and data provenance	2
3	System Architecture and End-to-End Workflow	4
3.1	Observation Alignment and Data Interface	4
3.2	Three-Dimensional Reference States and Camera Geometry	5
3.2.1	Reference-state construction	5
3.2.2	Effective camera estimation	6
3.3	Continuous Three-Dimensional Motion Model	8
3.3.1	Data split and prediction protocol	8
3.3.2	Model choice and implementation	8
3.3.3	Held-out evaluation	11
3.4	Editable Trajectories under a Fixed Camera	14
3.5	Scene Reconstruction and Conditional Video Generation	15
3.5.1	Shared background reconstruction	15
3.5.2	Condition construction and model selection	17
3.5.3	Visual evaluation	19
4	Main Limitations	21
5	Future Project Plan: Physics-Aware Generative Extension	22
6	Implementation references and asset notes	22
	References	23

1 Introduction and Literature Review

This project investigates how an editable three-dimensional baseball trajectory can be translated into a controlled visual result in a real broadcast view. The objective is to build a traceable workflow from observed pitches to modified motion and generated video. A change in the initial state or pitch condition should produce a coherent trajectory change while the camera, background, timing, and evaluation target remain fixed.

The workflow begins with broadcast video, reviewed two-dimensional ball locations, and Statcast parameters. After frame-time alignment, these sources are used to construct reference position-velocity states and estimate a local effective camera. A compact continuous-state model, following the Neural ODE formulation [1], predicts the ball’s three-dimensional motion over the visible flight. At inference time, the initial state and pitch-type condition can be edited. The predicted states are then projected into image coordinates, converted into visual conditions, and placed over a repaired broadcast background. ProPainter is used for background completion [4], while frozen AnimateDiff and SparseCtrl components provide video synthesis and structural conditioning [5, 6]. The resulting pipeline therefore connects data alignment, trajectory learning, camera projection, condition rendering, and video generation through explicit intermediate representations.

This project builds on several established research directions. TrackNet demonstrated the use of consecutive broadcast frames to locate small, fast-moving sports objects [2], and baseball-specific work has explored three-dimensional pitch reconstruction from single-view video [3]. In generative video research, motion trajectories have already been used to control object and camera motion [7, 8]. PhysCtrl further showed that three-dimensional, physics-grounded trajectories can drive controllable image-to-video generation [9]. The general connection among motion, camera geometry, and video synthesis is therefore not unique to this project.

The distinction of the present work is narrower and application-specific. It combines reviewed baseball observations, Statcast-derived reference states, an estimated broadcast camera, editable continuous-state rollouts, and fixed-scene video generation in one recorded experiment. Its modular interfaces allow trajectory, projection, conditioning, and generation errors to be examined separately. This is a system-level integration and evaluation contribution for a specific baseball broadcast setting, rather than a claim of a new tracking method, dynamics model, camera model, or video generation architecture.

2 Scope, assets, and data provenance

The dataset currently comprises 30 pitches thrown by Kyle Bradish in 2023: 10 four-seam fastballs (FF), 10 sliders (SL), and 10 curveballs (CU). Of these, 9 FF and 10 SL pitches were used for motion learning. FF_05 was excluded because of a data discontinuity, while the CU pitches contained events such as bounces and collisions that were outside the scope of the current model, although they remain potential targets for future work.

All source videos were recorded from a single local broadcast view at a resolution of 1280×720 pixels and a nominal frame rate of 60 fps. Only the short interval in which the ball was visible in flight was retained. The 19 usable FF/SL clips contain 25–30 frames each, corresponding to 0.400–0.484 s between their first and last retained frames. In total, the learning dataset contains 507 video frames, including 497 reviewed ball-center observations and 10 frames without a reliable observation. Actual container presentation timestamps were retained rather than assuming perfectly uniform $1/60$ s

frame spacing. Complete release, collision, rebound, contact, and catching events were not modeled.

I manually annotated the two-dimensional ball center (u, v) in the retained video frames and corrected a small number of ambiguous cases. Severe occlusion and widespread false detections were not common; the main visual difficulties were separating the white ball from white uniforms and identifying the ball near the moment of release. Missing observations were preserved as missing rather than filled by interpolation.

For motion learning, each timestamp is associated with a six-dimensional reference state

$$\mathbf{s}(t) = [x, y, z, v_x, v_y, v_z],$$

where (x, y, z) are the three-dimensional ball position in metres and (v_x, v_y, v_z) are the corresponding velocity components in metres per second. The world axes point toward catcher-right (x), toward the pitcher (y), and upward (z). These reference states were constructed from Statcast position, velocity, and acceleration parameters; they are not independent per-frame 3D measurements. Pitch type, FF or SL, is supplied as an additional categorical condition, implemented as a two-entry one-hot vector. Thus, the model uses six continuous state variables plus pitch type as a conceptual seventh variable. Elapsed video timestamps determine when the state is evaluated but are not treated as an additional state component.

Data	Source
Pitch trajectory data	Baseball Savant Statcast Search
Broadcast images	Frames extracted from pitch videos provided by Baseball Savant

Table 1: Data sources used in this project.

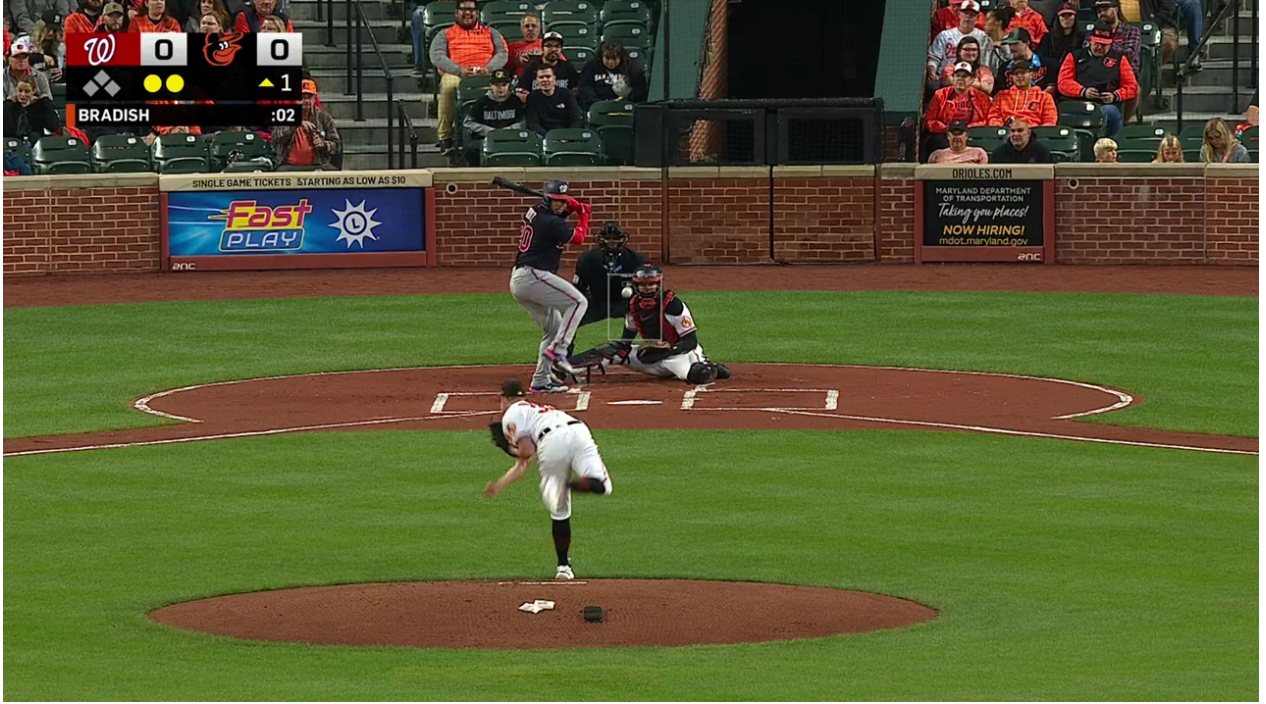
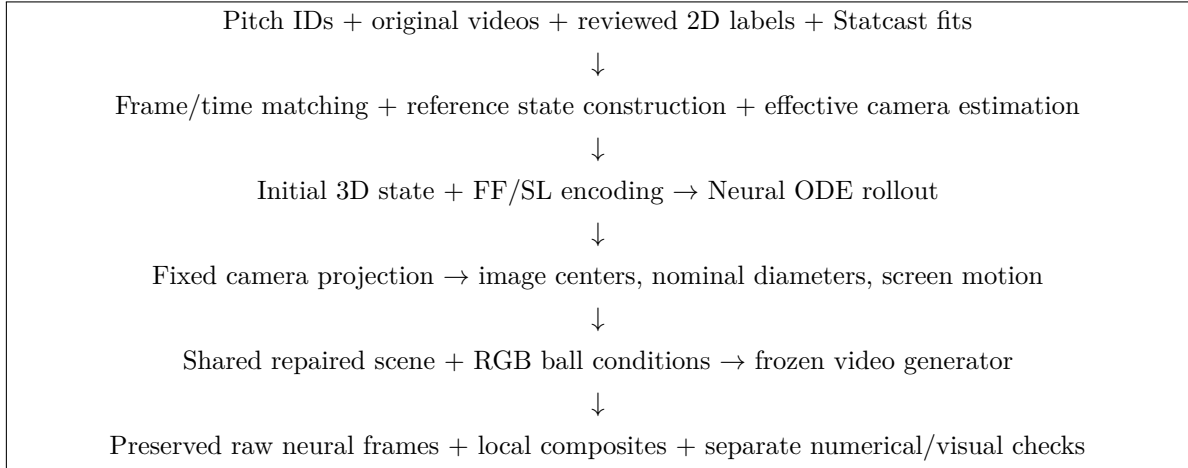


Figure 1: A 1280×720 broadcast frame from Kyle Bradish’s four-seam fastball to Keibert Ruiz on September 26, 2023. The pitch is designated FF_02 in our dataset.

3 System Architecture and End-to-End Workflow



3.1 Observation Alignment and Data Interface

Methods such as TrackNet use consecutive frames to detect small, fast-moving balls in broadcast sports video [2]. In this project, however, detector outputs were not treated as final ground truth. The motion experiment uses manually reviewed ball centers so that timing and projection errors can be examined without also introducing an unmeasured detector error.

Pipeline component	Main operation	Recorded output
Observation interface	Align reviewed ball centers with decoded video frames	Timestamped observations, source clips, and frame overlays
Reference geometry	Construct 3D reference states and estimate an effective camera	Aligned reference trajectories, camera parameters, and reprojection checks
Motion model	Learn continuous evolution of position and velocity	Data split, selected checkpoint, predicted trajectories, and held-out metrics
Trajectory control	Edit the initial state under a fixed model and camera	Five controlled trajectories, projected centers, sizes, and RGB conditions
Visual synthesis	Repair a shared scene and run a frozen video generator	Raw neural frames, local composites, and visual-quality checks

Table 2: The pipeline preserves an inspectable interface between observation, geometry, dynamics, and visual synthesis. Numerical and visual evaluations are reported separately.

The annotation interface stores playback seek times. Each seek time was matched to the decoded frame whose container presentation interval contains that time. Selecting the nearest first-frame-normalized OpenCV timestamp can return a different frame. Correcting this correspondence placed the FF_02 observations two frames earlier and the SL_02 observations one frame earlier than in the previous export. The ball-center coordinates themselves were not changed to improve the fit.

FF_02 contains 26 retained frames and 26 reviewed observations over 0.417078 s. SL_02 contains 28 frames, 27 reviewed observations, and one missing observation over 0.450444 s. The missing SL_02 entry had previously been interpolated and was therefore excluded from manual truth. During expansion to all 19 usable FF/SL pitches, six previously omitted manual points were restored. Five repeated seek times that referred to the same decoded frame were consolidated using the last actual manual coordinate while preserving the redundant source records.

The resulting dataset contains 507 frames, 497 reviewed observations, and 10 frames without a reliable observation. Both container timestamps and first-frame-normalized timestamps are retained. This avoids assuming that every frame is separated by exactly 1/60 s even though the videos have a nominal frame rate of 60 fps.

3.2 Three-Dimensional Reference States and Camera Geometry

3.2.1 Reference-state construction

Statcast reports release position together with velocity and acceleration parameters defined at specific reference locations. In particular, velocity at the $y = 50$ ft reference plane cannot be used directly as release velocity. The implementation solves the reference-to-release time relation and evaluates a consistent six-dimensional state at each video timestamp:

$$\mathbf{p}(t) = \mathbf{p}_{50} + \mathbf{v}_{50}t + \frac{1}{2}\mathbf{a}t^2, \quad (1)$$

$$\mathbf{v}(t) = \mathbf{v}_{50} + \mathbf{a}t. \quad (2)$$

The state is

$$\mathbf{s}(t) = [x, y, z, v_x, v_y, v_z],$$

where position is measured in metres and velocity in metres per second. The world origin is the back tip of home plate; x points toward catcher-right, y toward the pitcher, and z upward. The Statcast acceleration is a fitted net acceleration and already includes gravity. These states are reference trajectories derived from a whole-pitch fit, not independent per-frame 3D measurements.

Baseball reconstruction studies have also mapped tracked single-view image coordinates directly into three-dimensional trajectories [3]. The present project follows a different protocol: reviewed 2D centers constrain timing and projection, whereas the 3D states are constructed from Statcast parameters. The experiment therefore evaluates short-horizon state evolution under a supplied reference state; it does not reconstruct an unknown 3D pitch from video alone.

3.2.2 Effective camera estimation

The reference trajectory is connected to the broadcast image through an explicit pinhole projection:

$$\mathbf{X}_c = R\mathbf{X}_w + T, \quad (3)$$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim K\mathbf{X}_c, \quad t_{\text{world}} = t_{\text{video}} - t_{\text{origin}}. \quad (4)$$

Approximate field correspondences, static-background ORB/RANSAC matches, Statcast reference positions, and reviewed ball centers constrain a bounded weighted least-squares fit. The explicit projection was chosen because it provides a direct, inspectable mapping between a predicted 3D state and its expected image location. This is an effective local camera model, not recovered broadcast-camera metadata or a unique hardware calibration.

The base FF fit has principal point (640, 360) px, effective focal length approximately 22,420.6 px, and fitted camera center (−6.301, 173.355, 14.342) m. The SL view transfer contains approximately 1.16% scale change and a small image shift. Alternative focal lengths near 12,000 and 28,000 px can fit the same short observation window with different camera distances. The fitted camera is therefore locally useful but not uniquely identifiable.

fig:projection



Figure 2: Reprojection of a Statcast-derived 3D reference state onto the original FF_02 broadcast frame. The yellow cross marks the projected Statcast reference position, and the green circle marks the manually reviewed ball center; their separation in this frame is approximately 0.86 px. This close local agreement supports use of the projection within the sampled view, but does not establish unique physical camera parameters.

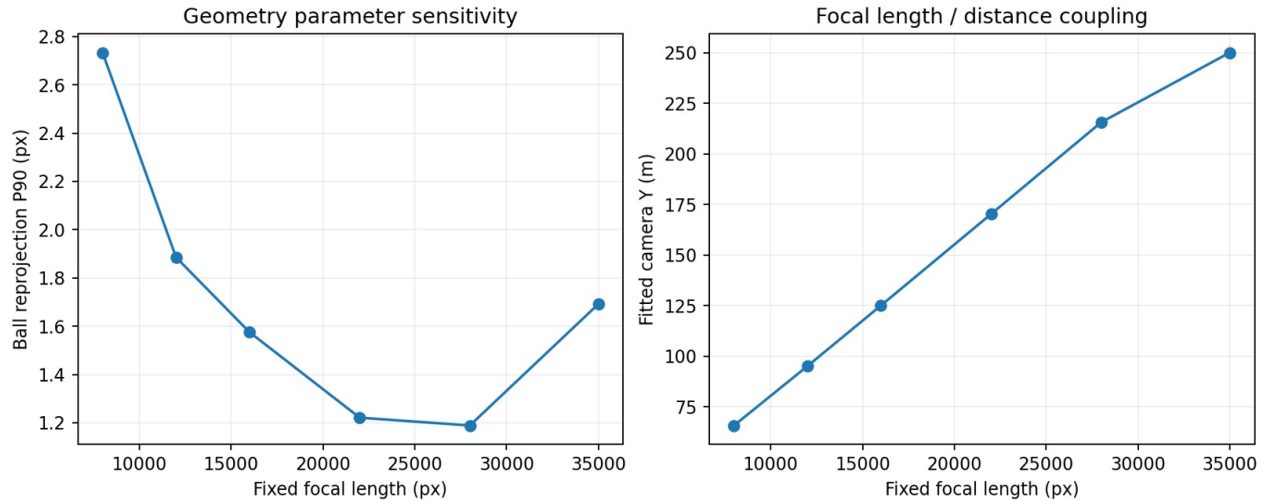


Figure 3: Camera-parameter sensitivity analysis. Similar reprojection errors are obtained with materially different focal-length and camera-distance combinations, demonstrating that the short ball trajectory does not uniquely determine the broadcast camera.

3.3 Continuous Three-Dimensional Motion Model

3.3.1 Data split and prediction protocol

Pitches, rather than neighboring frames, define the data split:

- Train (13): FF01, FF02, FF04, FF06, FF07, FF08; SL01, SL02, SL04, SL05, SL06, SL07, SL09.
- Validation (2): FF10, SL10.
- Test (4): FF03, FF09, SL03, SL08.

For each pitch, the observation prefix contains $\lceil N_{\text{manual}}/3 \rceil$ reliable ball centers. Only that prefix is used to refine the video-to-world time offset. The reference state at the final prefix frame initializes the prediction.

The model receives the initial position, initial velocity, FF/SL condition, and elapsed timestamps. It does not receive future states, future image centers, fitted acceleration, or pitch identity. The predicted trajectory is integrated forward without future-state teacher forcing.

The initial state remains an **oracle reference state** because it is obtained from the whole-pitch Statcast fit. The held-out experiment is consequently not a video-only forecast and does not provide independent 3D physical validation.

3.3.2 Model choice and implementation

A Neural ODE was selected because the desired output is a continuous position-velocity trajectory evaluated at the actual, slightly irregular video timestamps [1]. Compared with a frame-indexed recurrent or Transformer model, the ODE provides a compact initial-value formulation and allows the same learned dynamics to be evaluated after controlled changes to the initial state. Compared with an analytic constant-acceleration equation, it can represent state- and pitch-dependent acceleration, although the present experiment does not demonstrate that this flexibility improves accuracy.

The model evolves

$$\dot{\mathbf{p}} = \mathbf{v}, \quad \dot{\mathbf{v}} = (0, 0, -9.81) + f_{\theta}(\text{normalize}(\mathbf{s}), \mathbf{c}), \quad (5)$$

where $\mathbf{c}$ is a two-entry FF/SL encoding. The acceleration network is $8 \rightarrow 32 \rightarrow 32 \rightarrow 3$, with two Tanh hidden layers, an output scale of 10, and 1,443 trainable parameters. Explicit gravity, train-only normalization, and a small final-layer initialization keep the initial integration behavior controlled.

The differentiable RK4 solver uses internal steps of at most $1/240$ s and shortens the final step to evaluate each actual timestamp. Training minimizes

$$\mathcal{L} = \text{MSE}(\hat{\mathbf{p}}_{\text{norm}}, \mathbf{p}_{\text{norm}}) + 0.25 \text{MSE}(\hat{\mathbf{v}}_{\text{norm}}, \mathbf{v}_{\text{norm}}) + 10^{-6} \text{mean}(\theta^2). \quad (6)$$

Each pitch receives equal weight. Adam uses learning rate 10^{-3} , gradient clipping at norm 10, and seed 101. The run stopped at epoch 175 and selected epoch 75 using validation loss before test

evaluation. Training required approximately 11 s on one CPU thread. The camera remained fixed and no image-space projection loss was used.

Related physics-grounded video work has used substantially larger generative dynamics models trained on simulated 3D motion [9]. The present model is deliberately smaller because only 19 usable pitches and two pitch types are available. It should be interpreted as a local continuous-state pilot, not as a general baseball physics model.

Executed source: acceleration network + RK4 step

```

11 class BallODE(nn.Module):
12     def __init__(self, normalization):
13         super().__init__()
14         self.register_buffer('state_mean', torch.tensor(normalization['state_mean'], dtype=torch.float64))
15         self.register_buffer('state_std', torch.tensor(normalization['state_std'], dtype=torch.float64))
16         self.register_buffer('gravity', torch.tensor([0, 0, -9.81], dtype=torch.float64))
17         self.register_buffer('acceleration_scale', torch.tensor(10., dtype=torch.float64))
18         self.network = nn.Sequential(nn.Linear(8, 32), nn.Tanh(), nn.Linear(32, 32), nn.Tanh(), nn.Linear(32,
19 3)).double()
20         # Start with a small non-gravity output; every force correction is learned.
21         nn.init.normal_(self.network[-1].weight, mean=0, std=.01)
22         nn.init.zeros_(self.network[-1].bias)
23
24     def non_gravity_acceleration(self, state, pitch_onehot):
25         normalized = (state-self.state_mean)/self.state_std
26         return self.network(torch.cat([normalized, pitch_onehot], dim=-1))*self.acceleration_scale
27
28     def forward(self, state, pitch_onehot):
29         return torch.cat([state[..., 3:], self.gravity+self.non_gravity_acceleration(state, pitch_onehot)],
30 dim=-1)
31
32 def rk4_step(model, state, pitch_onehot, dt):
33     h = dt[:, None]
34     k1 = model(state, pitch_onehot)
35     k2 = model(state+h*k1/2, pitch_onehot)
36     k3 = model(state+h*k2/2, pitch_onehot)
37     k4 = model(state+h*k3, pitch_onehot)
38     return state+h*(k1+2*k2+2*k3+k4)/6

```

Figure 4: Executed Neural ODE source showing the acceleration network, explicit gravity, and RK4 integration. The compact architecture matches the small dataset but does not by itself guarantee physically correct aerodynamics.

Executed source: exact irregular timestamp integration

```

40 def integrate(model, initial_state, pitch_onehot, elapsed_times, max_step=1/240):
41     """Batched exact timestamp sampling, shortened final steps, padded times allowed."""
42     assert elapsed_times.ndim == 2 and initial_state.shape[-1] == 6
43     assert torch.all(elapsed_times[:, 0] == 0)
44     state = initial_state
45     states = [state]
46     for index in range(1, elapsed_times.shape[1]):
47         dt = elapsed_times[:, index] - elapsed_times[:, index-1]
48         assert torch.all(dt >= 0)
49         steps = max(1, math.ceil(float(dt.max())/max_step-1e-12))
50         for _ in range(steps):
51             state = rk4_step(model, state, pitch_onehot, dt/steps)
52             states.append(state)
53     return torch.stack(states, dim=1)

```

Figure 5: Executed solver code for evaluating irregular presentation timestamps with shortened final integration steps. This implementation prevents timestamp rounding from being absorbed into the learned motion.

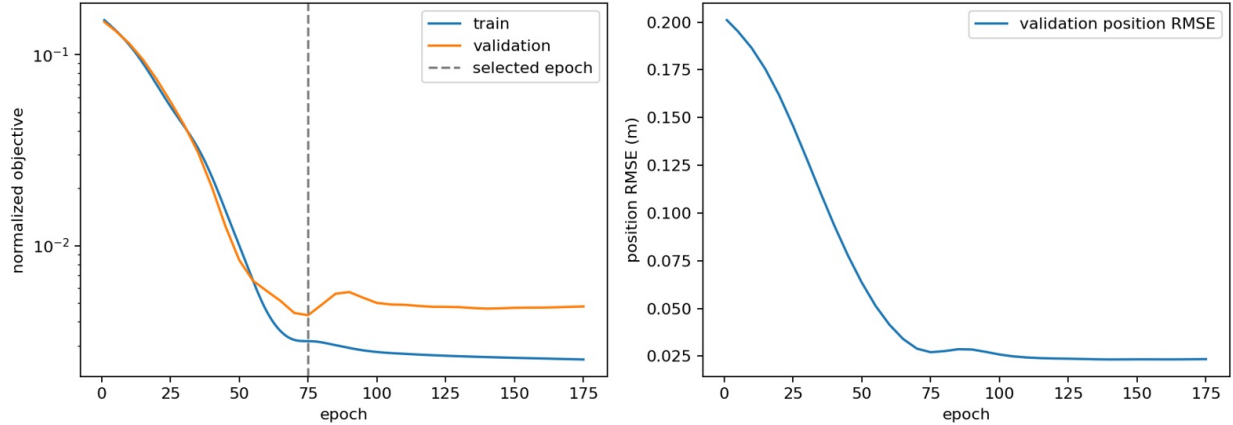


Figure 6: Sensitivity of the fitted camera to focal length. In the left panel, each focal length is fixed while the remaining camera parameters are refitted; several substantially different focal lengths produce similarly small ball-center reprojection errors. The right panel shows the corresponding increase in fitted camera distance, indicating that focal length and distance compensate for one another. The estimated camera is therefore suitable as a local projection model, but its parameters should not be interpreted as a unique physical calibration.

3.3.3 Held-out evaluation

Metrics exclude the initial seed state. The test set contains 70 future state samples and 69 reviewed future image observations; the missing observation is not replaced with interpolated truth.

Check	Recorded result
Position RMSE against reference	0.024834 m (2.48 cm)
Velocity RMSE against reference	0.244268 m/s
Predicted projection to manual center, median	1.2812 px
Predicted projection to manual center, P90	3.6625 px
Predicted projection to manual center, maximum	7.9789 px
Reference projection to manual center, median	1.0899 px
Reference projection to manual center, P90	2.7265 px

Table 3: Held-out errors pooled by frame. P90 is the 90th percentile of valid center errors, not the maximum. The reference-projection rows measure alignment consistency and are not an independent dynamics baseline.

FF_09 contributes the largest late-window projection deviations, and its failure frames are retained in the reported maximum. Step-halving and analytic solver checks show that numerical integration error is negligible relative to the combined motion, reference, camera, and timing errors. All predicted points remain in front of the camera during evaluation.

The results support short-horizon state evolution within the fitted reference family. They do not demonstrate learned aerodynamic laws, recovered spin, or independent 3D truth. Because the targets follow Statcast constant-acceleration fits, the network may mainly approximate that fitted trajectory family. No analytic dynamics baseline, recurrent model, or model-size ablation was run, so the results do not establish superiority of the Neural ODE over those alternatives.

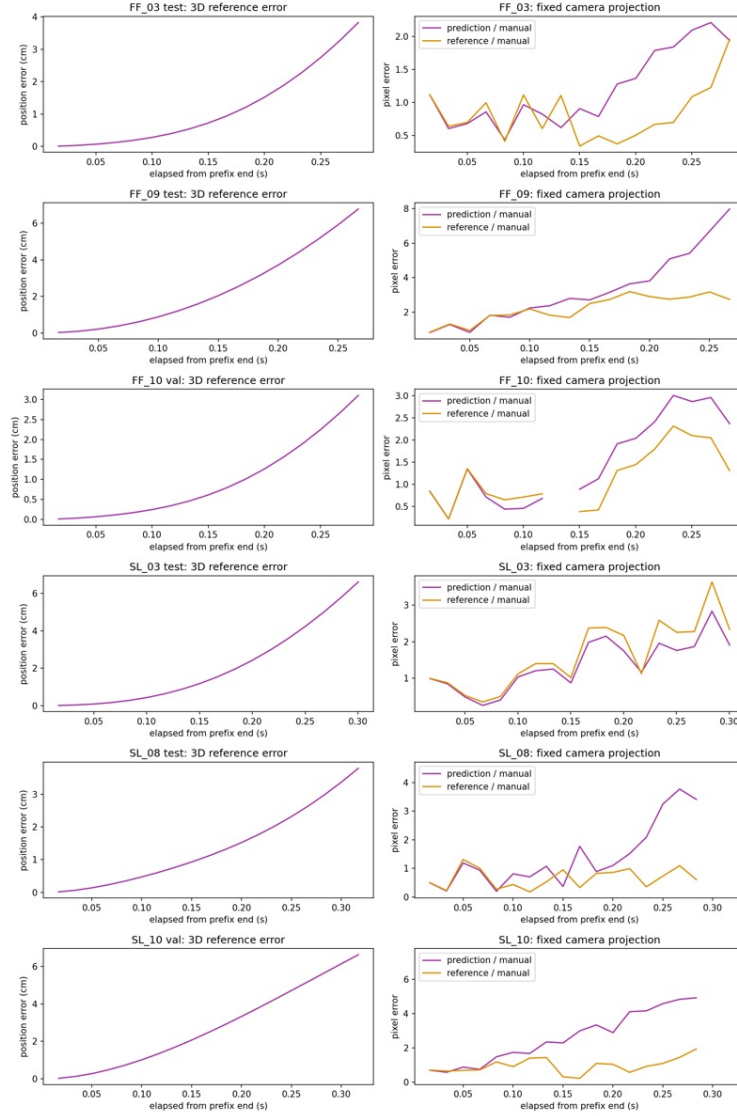


Figure 7: Held-out state-space and image-space errors. The larger image errors near the end of some trajectories include contributions from motion prediction, effective-camera projection, timestamp alignment, and the fitted reference itself.

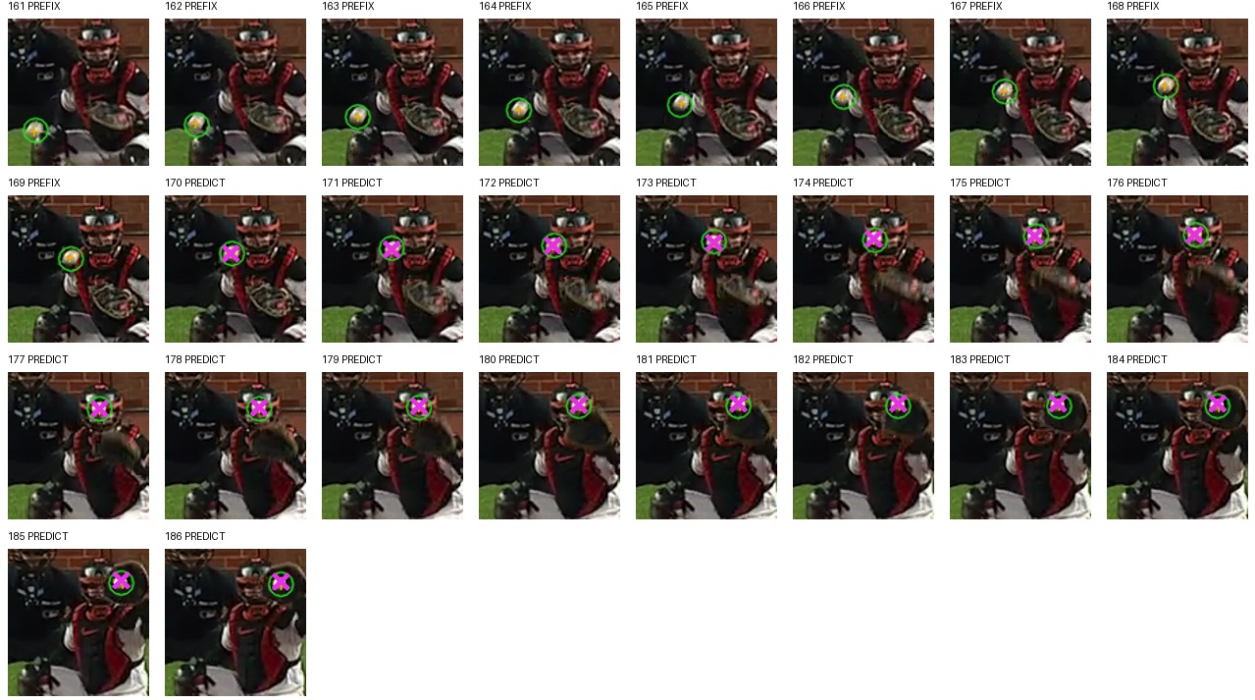


Figure 8: Sequential crops from the held-out FF_03 broadcast video. The green circle denotes the manually reviewed ball center, the yellow cross denotes the projected Statcast-derived reference position, and the magenta X denotes the projected Neural ODE prediction. Frames 161–169 form the observation prefix, with frame 169 providing the initial reference state for the rollout. Frames 170–186 are future prediction frames evaluated without future-state teacher forcing. For this pitch, the final three-dimensional position error is approximately 3.83 cm, while the projected prediction remains within 2.00 px at the 90th percentile of the reviewed future centers. The seed frame is excluded from these future-error statistics.

3.4 Editable Trajectories under a Fixed Camera

The editable window begins at FF_02 decoded frame 191, corresponding to the last reliable prefix observation:

$$\begin{aligned}\mathbf{p}_0 &= (-0.327831, 10.911661, 1.701158) \text{ m}, \\ \mathbf{v}_0 &= (0.658690, -41.361149, -1.879336) \text{ m/s}.\end{aligned}$$

This state lies within the visible flight and is not the release state. The control window contains 16 frames spanning 0.250244 s between the first and last presentation times, or 0.266933 s including the final frame duration. Model weights, timestamps, background, and the FF_02 camera are held fixed.

Case	Edit relative to FF base	Endpoint image displacement
FF base	No edit	0 px
SL variant	Same position; SL02 prefix velocity and SL encoding	86.98 px
FF slower	Multiply velocity by 0.98	3.08 px
FF world- x edit	Add 0.35 m/s to v_x	11.83 px
FF upward edit	Add 0.30 m/s to v_z	9.27 px

Table 4: Controlled initial-state edits under one fixed model and camera. The SL variant changes both velocity and pitch encoding and is therefore not a type-only causal comparison. The world- x edit appears toward image-left in this camera, despite the historical code label `ff_rightward`.

No endpoint was forced to match the original catch location. The edited components remain within their recorded training ranges, but component-wise inclusion does not prove that the combined state is physically realizable. These are forward initial-value experiments, not inverse solutions for arbitrary target locations.

The fixed camera converts each state into an image center, screen velocity, and depth-based nominal diameter. For physical ball diameter $D = 0.073$ m,

$$d \simeq \frac{fD}{Z_c}.$$

The FF nominal diameter changes from approximately 10.04 to 9.45 px across the window. This small change is consistent with the fitted perspective geometry, although a small set of reliable ball-size annotations would be required to validate the effective scale and distinguish physical diameter from motion blur.

Trajectory-conditioned video generation is already well established. Motion Prompting uses sparse or dense trajectories as motion conditions [7], while MotionAgent combines object trajectories and camera extrinsics through an optical-flow representation [8]. The present project does not introduce a new trajectory-control mechanism. Its narrower role is to obtain the control trajectory from an editable baseball state model and project it through a fixed, application-specific camera.

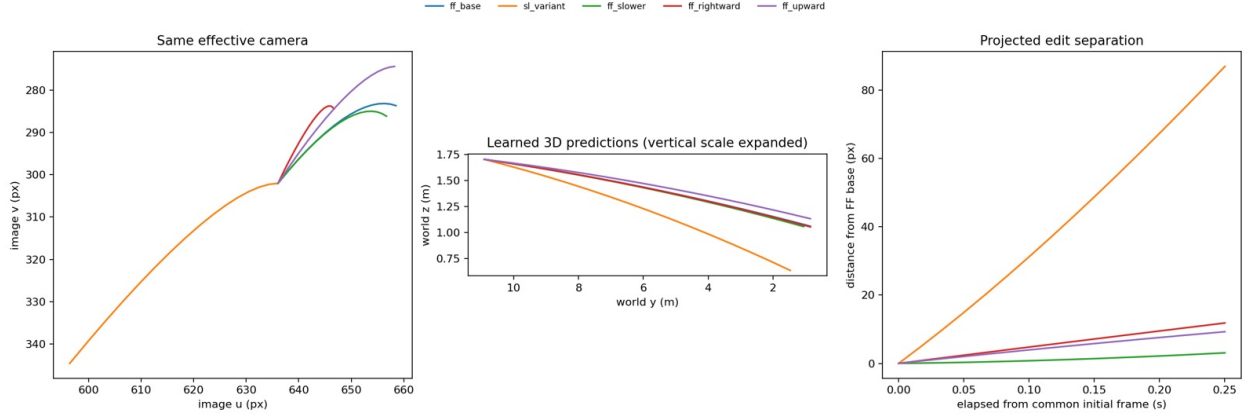


Figure 9: Controlled trajectory edits starting from the same visible-flight position in FF_02, with the Neural ODE weights, timestamps, and effective camera held fixed. The left panel shows the resulting image-plane trajectories, where u increases to image-right and v increases downward. The middle panel shows the corresponding predicted motion in the world y - z plane. The right panel reports the Euclidean image-plane separation from the unedited FF baseline over time. At the final frame, the SL variant, 2%-slower FF, world- x edit, and upward edit are separated from the FF baseline by 86.98, 3.08, 11.83, and 9.27 px, respectively. These results confirm that initial-state edits propagate through the learned model and fixed-camera projection. They do not establish that the edited trajectories are independently validated physical counterfactuals; in particular, the SL variant changes both initial velocity and pitch-type encoding.

3.5 Scene Reconstruction and Conditional Video Generation

3.5.1 Shared background reconstruction

A direct sprite overlay requires removal of the observed ball from the source frames. Simple Telea inpainting produced moving smears near the helmet and strike-zone line. ProPainter was therefore selected because its propagation and temporal-attention mechanisms use information across neighboring frames rather than filling every frame independently [4].

The official ProPainter implementation at commit `e870e79321c31b733e2031af5aa2fb1fe3ac7eec` was run on 76 context frames: 30 before the 16-frame target, the target frames, and 30 after it. Processing used a 256×256 crop, old-ball masks of radius 12 px with 4 px dilation, neighbor length 10, reference stride 5, subvideo length 80, and fp16. The repair required approximately 20.40 s.

Only pixels inside the dilated old-ball masks were written back to the full-resolution frames. The same repaired background is used for every trajectory condition. Inspection shows better preservation of the helmet and strike-zone structure than the earlier frame-independent repair, but the reconstructed pixels remain estimates rather than observed ground truth.



Figure 10: Original and ProPainter-repaired crops at selected target times. The old ball is removed and nearby helmet and strike-zone structure is better preserved than with frame-independent filling. The comparison demonstrates local visual improvement, not recovery of the true hidden pixels.

3.5.2 Condition construction and model selection

Five real RGBA ball sprites are blended in premultiplied-alpha space along each projected trajectory. Perspective scaling follows camera depth and empirical blur orientation follows projected screen motion. The conditions do not encode recovered spin, seam orientation, or measured shutter exposure.

The visual generator combines a Realistic Vision V5.1 noVAE image model, AnimateDiff v1-5-3, RGB SparseCtrl, SD VAE ft-MSE, and the v1-5-3 motion LoRA. AnimateDiff was chosen because it adds a reusable temporal motion module to an existing image diffusion model [5]. SparseCtrl accepts image-based structural conditions while leaving the pretrained video model frozen [6]. Freezing the generator is appropriate for this pilot because the baseball dataset is too small to train or fine-tune a video model reliably.

Unlike MotionAgent, which constructs optical flow from object and camera motion [8], and PhysCtrl, which drives generation with learned physics-grounded 3D point trajectories [9], this implementation projects the predicted baseball trajectory into dense RGB condition frames. This choice simplifies integration with the available SparseCtrl checkpoint, but it can also encourage the generator to copy the supplied ball appearance. The experiment therefore evaluates whether the complete pipeline runs coherently; it does not test whether RGB conditioning is superior to sparse trajectories, optical flow, or direct 3D control.

Exact model revisions are pinned in `configs/generative_models.json`. Each case uses a common 384×384 region of interest, 16 dense RGB conditions, 20 inference steps, seed 101, guidance scale 6, control scale 1, and CPU offload. The environment uses Python 3.12.3, Torch 2.8.0+cu128, Diffusers 0.39.0, Transformers 4.57.6, and Accelerate 1.15.0.

Generation and media export required approximately 19.32 s for FF and 18.60 s for SL on one A40 GPU, with approximately 5.03 GiB allocated peak VRAM.

Executed source: pinned frozen models + dense RGB controls

```

48     if importlib.util.find_spec('hf_transfer') is None: os.environ['HF_HUB_ENABLE_HF_TRANSFER']='0'
49     import torch, diffusers
50     from PIL import Image
51     from diffusers import AnimatedDiffSparseControlNetPipeline, MotionAdapter, SparseControlNetModel, AutoencoderKL,
    DPMSolverMultistepScheduler
52     subprocess.run([sys.executable, '-m', 'pip', 'freeze'], stdout=(out/'requirements_resolved.txt').open('w'), check
    =True)
53     models, revisions=job['models'], job['revisions']; common={'torch_dtype': torch.float16}
54     motion=MotionAdapter.from_pretrained(models['motion'], revision=revisions['motion'], **common)
55     control=SparseControlNetModel.from_pretrained(models['control'], revision=revisions['control'], **common)
56     vae=AutoencoderKL.from_pretrained(models['vae'], revision=revisions['vae'], **common)
57     pipe=AnimatedDiffSparseControlNetPipeline.from_pretrained(models['base'], revision=revisions['base'], motion_ad
    apter=motion, controlnet=control, vae=vae, **common)
58     pipe.scheduler=DPMSolverMultistepScheduler.from_config(pipe.scheduler.config, beta_schedule='linear', algorith
    m_type='dpmsolver++', use_karras_sigmas=True)
59     pipe.load_lora_weights(models['lora'], revision=revisions['lora'], adapter_name='motion_lora')
60     pipe.vae.enable_slicing(); pipe.enable_model_cpu_offload()
61     dump(out/'environment.json', {'python': platform.python_version(), 'torch': torch.__version__, 'diffusers': diffu
    sers.__version__,
62         'models': models, 'revisions': revisions, 'gpu': torch.cuda.get_device_name(0), 'job': job})
63     for name in job['primary_cases']:
64         target=out/name; assert not target.exists(), 'Use a fresh results directory, never overwrite a prior
    neural run'
65         target.mkdir(); conditions=[Image.open(out/'conditions'/name/f'{i:03d}.png').convert('RGB') for i in
    range(16)]
66         started=time.perf_counter(); torch.cuda.reset_peak_memory_stats()
67         try:
68
69             frames=pipe(prompt=job['prompt'], negative_prompt=job['negative_prompt'], height=384, width=384, num_frames=16,
                num_inference_steps=20, guidance_scale=6., conditioning_frames=conditions, controlnet_frame_indices
    =list(range(16)),
70                 controlnet_conditioning_scale=1., generator=torch.Generator('cpu').manual_seed(101)).frames[0]
71             assert len(frames)==16
72             export_outputs(ROOT, name, frames)
73
74         dump(target/'run.json', {'status': 'generated_visual_ball_checks_pending', 'elapsed_s': time.perf_counter()-started,
                'peak_vram_GiB': torch.cuda.max_memory_allocated()/2**30, 'seed': 101, 'steps': 20,
75                 'raw_frames_preserved': True, 'no_program_ball_added_to_neural_output': True, 'first_frame_exact_pre
    servation': False})
76         except Exception as error:

```

Figure 11: Executed source for pinned model loading and dense RGB-conditioned generation. All network weights are frozen; the experiment performs inference rather than training a new baseball video model.

```

Executed source: preserve raw pixels + local neural compositing

93     mask=cv2.GaussianBlur(mask,(9,9),1.8);mask=np.clip(mask,0,1)
94
95     generated=cv2.resize(cv2.cvtColor(np.array(frame),cv2.COLOR_RGB2BGR),(w,h),interpolation=cv2.INTER_CUBIC)
96     region=bg[cy:cy+h,cx:cx+w].astype(float);alpha=mask[cy:cy+h,cx:cx+w,None]
97     result=bg.copy();result[cy:cy+h,cx:cx+w]=np rint(region*(1-alpha)+generated*alpha).clip(0,255).astype(np.uint8)
98     assert not np.any(np.any(result!=bg,axis=2)&(mask==0))
99     cv2.imwrite(str(full/f'{i:04d}.png'),result);cv2.imwrite(str(masks/f'{i:04d}.png'),np rint(mask*255).ast
100     ype(np.uint8))
101     times=np.array(job['elapsed_times_s']);last=job['last_frame_duration_s']
102     for folder,label in [(raw,'raw'),(full,'composite')]:
103         for factor,speed in
104             [(1,'normal'),(5,'slow_5x')]:encode(folder,times*factor,last*factor,out/f'{label}_{speed}.mp4')

```

Figure 12: Executed compositing code. Raw neural frames are saved before local compositing, and generated pixels are restricted to a softened projected-ball support region. No programmatic ball is painted over the neural output after generation.

3.5.3 Visual evaluation

Raw neural frames are preserved unchanged. Local composites insert generated pixels from the projected-ball region into the shared repaired scene. This reduces unrelated changes to the player and strike-zone graphics, but it may also hide off-trajectory artifacts. Raw outputs and composites must therefore be evaluated separately.

All 32 raw frames and 32 composites were inspected. The SL ball is identifiable in all 16 frames. The FF ball is identifiable in the first 12 frames; in the final four, its predicted location overlaps bright helmet structure and the generated ball cannot be separated reliably. These four frames are classified as uncertain rather than successful detections.

No clear duplicate or completely absent ball was found in the remaining frames. However, the output retains substantial similarity to the dense RGB condition. Shape, brightness, and apparent blur vary, while some raw frames also modify player edges and strike-zone details. Since no physical foreground-depth or occlusion mask was provided, fusion with the helmet is a generation failure, not a valid reconstruction of physical occlusion.

Independent generated-ball centers have not been annotated. Consequently, the report does not claim an all-frame center error, generated-ball P90, recovered spin, or physically accurate occlusion.

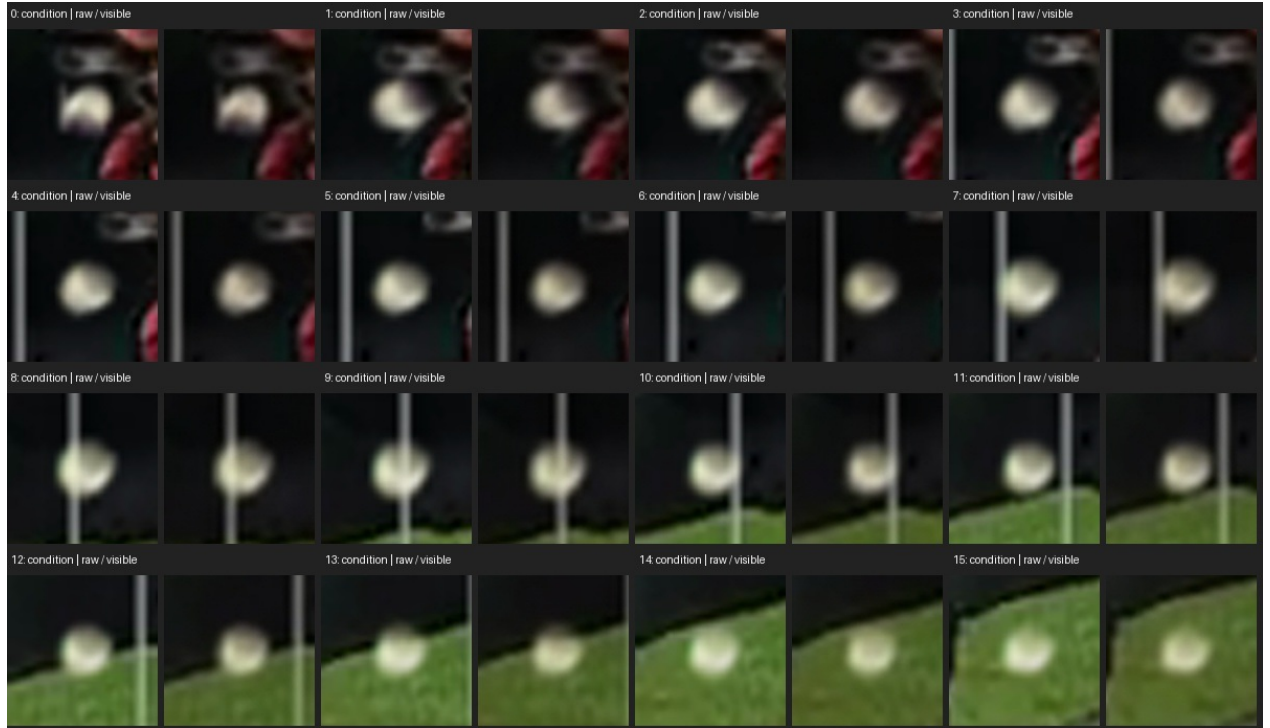


Figure 13: All 16 SL ball crops, with the RGB condition on the left and raw neural output on the right. The ball remains identifiable throughout, but similarity to the dense condition indicates that the result may rely strongly on condition copying. Crop centers are requested positions, not independently measured generated centers.

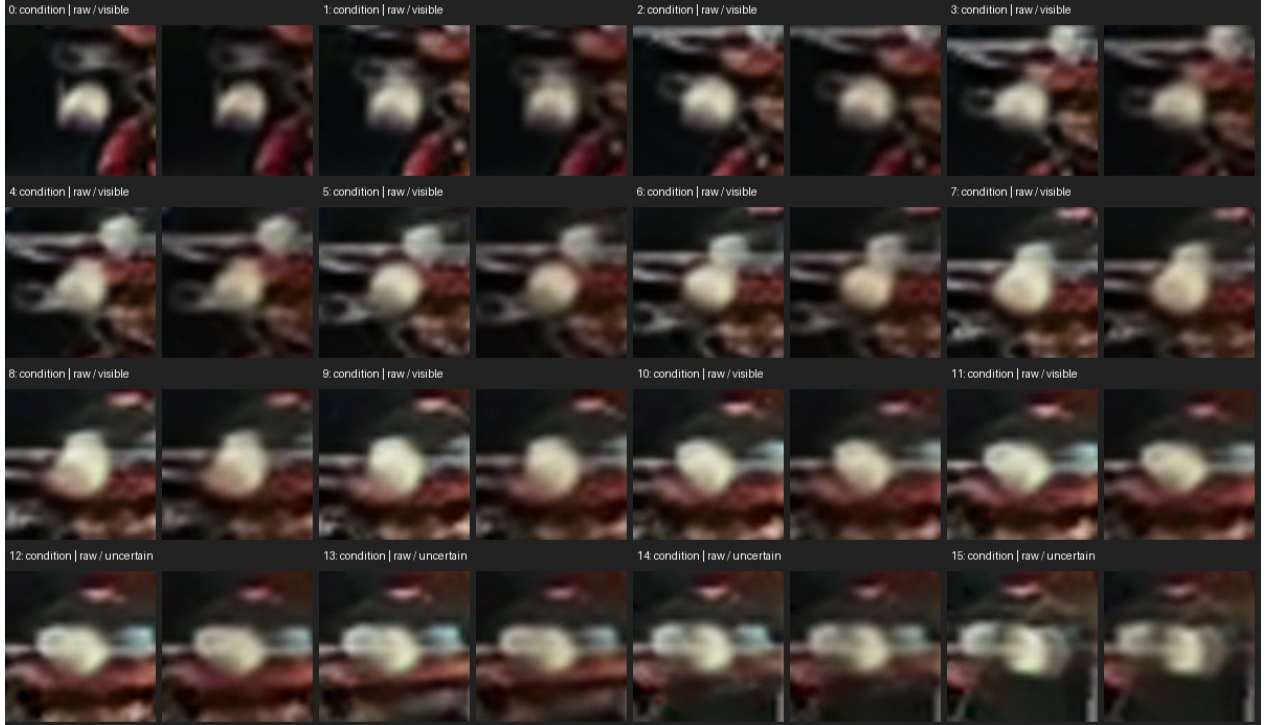


Figure 14: All 16 FF ball crops. The first 12 contain an identifiable ball; the final four overlap helmet highlights and do not provide reliable independent ball locations. The failure frames are retained rather than removed from the evaluation.

4 Main Limitations

1. **Dependence on fitted reference states.** Both the initial states and the training targets are derived from whole-pitch Statcast fits rather than independent per-frame 3D measurements. The system also does not infer a complete initial 3D state directly from the observed video prefix. Consequently, agreement with the reference trajectory cannot be interpreted as independent validation of the true physical motion.
2. **Limited data and evaluation scope.** The motion model uses only 19 pitches from one pitcher, two pitch types, and closely related broadcast views. Evaluation covers only short visible-flight windows of approximately 0.4–0.5 s, while video generation covers 16 frames. The results therefore do not establish generalization across pitchers, sessions, camera configurations, or complete pitch events such as release, catching, collision, and rebound.
3. **No established advantage over alternative motion models.** The Neural ODE was not compared with constant-acceleration continuation, physics-informed drag/lift models, recurrent networks, or other trajectory predictors. In addition, the main FF/SL comparison changes both the initial velocity and the pitch-type encoding. It therefore cannot isolate a pitch-type effect or establish that the edited trajectories are physically valid counterfactuals.
4. **Camera parameters are not uniquely identifiable.** Different focal-length and camera-distance combinations produce similarly small reprojection errors over the short observed trajectory. The estimated camera is useful as a local projection model for the sampled broadcast view, but its parameters should not be interpreted as a unique physical calibration or applied

directly to arbitrary camera views.

5. **Visual generation remains qualitatively evaluated.** The generated videos were checked mainly for ball presence and approximate motion direction. Independent generated-ball centers were not annotated, and no validated detector was used to measure trajectory accuracy. Dense RGB conditions may also be copied by the generator, while the final FF frames show unresolved fusion with helmet highlights. The current results therefore demonstrate pipeline execution, not quantitatively validated trajectory following, realistic spin, or physically correct occlusion.

5 Future Project Plan: Physics-Aware Generative Extension

These directions are proposals rather than completed results. The next phase can follow the central idea of the Chan group’s NewtonGen and SeeU frameworks: represent dynamics explicitly before using them to guide visual generation. NewtonGen separates learned Newtonian state evolution from a pretrained video generator, while SeeU follows a 2D-to-4D-to-2D pipeline in which scene motion is reconstructed, evolved continuously, and reprojected for generation [10, 11]. For baseball, the immediate goal should not be full 4D scene reconstruction, but a more reliable physical interface. Initial 3D states should first be estimated from observed video prefixes and evaluated against independent multi-view evidence where available; the Neural ODE should then be compared with constant-acceleration and drag/lift baselines on additional pitchers, pitch types, sessions, and cameras. The current continuous-flight model can subsequently be extended to a hybrid event model,

$$\text{flight dynamics} \rightarrow \text{contact event} \rightarrow \mathbf{s}^+ = J(\mathbf{s}^-, \mathbf{n}, \text{surface/object}) \rightarrow \text{flight dynamics},$$

where the jump map J represents the velocity and spin change caused by a bounce, bat contact, glove contact, or another collision. Event time, contact normal, surface identity, restitution, and friction should be recorded explicitly rather than forcing discontinuous motion into a single smooth ODE. The resulting 3D states can then be projected into position, size, visibility, motion-blur, and occlusion conditions for a frozen or lightly adapted video generator. Evaluation should remain independent of those conditions by measuring ball presence, generated center error, event timing, post-contact direction, background stability, and occlusion order in the raw generated frames. This extension would connect the Chan group’s physics-before-generation philosophy to a baseball-specific setting while directly addressing the present project’s dependence on fitted references, limited data, absence of model comparisons, and incomplete visual evaluation.

6 Implementation references and asset notes

- Statcast field descriptions: <https://baseballsavant.mlb.com/csv-docs>.
- ProPainter official code: <https://github.com/sczhou/ProPainter>.
- Diffusers AnimateDiff/SparseCtrl interface: <https://huggingface.co/docs/diffusers/api/pipelines/animatediff>.
- Exact model repositories and immutable revisions: `configs/generative_models.json` in the Git package.

References

- [1] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural Ordinary Differential Equations,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [2] Y.-C. Huang, I.-N. Liao, C.-H. Chen, T.-U. İk, and W.-C. Peng, “TrackNet: A Deep Learning Network for Tracking High-Speed and Tiny Objects in Sports Applications,” in *Proceedings of the International Conference on Applied System Innovation*, 2019.
- [3] J. Hsieh, “Neural Network-Based Tracking and 3D Reconstruction of Baseball Pitch Trajectories from Single-View 2D Video,” *arXiv preprint arXiv:2405.16296*, 2024.
- [4] S. Zhou, C. Li, K. C. K. Chan, and C. C. Loy, “ProPainter: Improving Propagation and Transformer for Video Inpainting,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.
- [5] Y. Guo, C. Yang, A. Rao, Z. Liang, Y. Wang, Y. Qiao, M. Agrawala, D. Lin, and B. Dai, “AnimateDiff: Animate Your Personalized Text-to-Image Diffusion Models without Specific Tuning,” in *International Conference on Learning Representations*, 2024.
- [6] Y. Guo, C. Yang, A. Rao, M. Agrawala, D. Lin, and B. Dai, “SparseCtrl: Adding Sparse Controls to Text-to-Video Diffusion Models,” in *European Conference on Computer Vision*, 2024.
- [7] D. Geng, C. Herrmann, J. Hur, F. Cole, S. Zhang, T. Pfaff, T. Lopez-Guevara, Y. Aytar, M. Rubinstein, C. Sun, O. Wang, A. Owens, and D. Sun, “Motion Prompting: Controlling Video Generation with Motion Trajectories,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [8] X. Liao, X. Zeng, L. Wang, G. Yu, G. Lin, and C. Zhang, “MotionAgent: Fine-Grained Controllable Video Generation via Motion Field Agent,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025.
- [9] C. Wang, C. Chen, Y. Huang, Z. Dou, Y. Liu, J. Gu, and L. Liu, “PhysCtrl: Generative Physics for Controllable and Physics-Grounded Video Generation,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [10] Y. Yuan, X. Wang, T. Wickremasinghe, Z. Nadir, B. Ma, and S. H. Chan, “NewtonGen: Physics-Consistent and Controllable Text-to-Video Generation via Neural Newtonian Dynamics,” in *International Conference on Learning Representations*, 2026.
- [11] Y. Yuan, T. Wickremasinghe, Z. Nadir, X. Wang, Y. Chi, and S. H. Chan, “SeeU: Seeing the Unseen World via 4D Dynamics-Aware Generation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.